One Service, Two Deployment Environments

CS 454/554 Midterm Mini-Project — Technical Report

CS 454/554 — Introduction to Cloud Computing, Fall 2026 (graduate section)

October 2026 • Repository:
https://github.com/RemyMartinRosenhoover/CS-554-MidTerm

1 Deployment overview

One small HTTP service runs in two environments; this report records how each manages it. Everything sits in four layers (Figure 1). The physical host is a Windows 11 workstation running VMware Workstation 17.6.1. Inside it, a virtual machine named `cs554-vm` runs Ubuntu Server 24.04.5 with 2 vCPUs, 4 GiB of RAM, and a 25 GiB disk, attached to VMware's NAT network `192.168.197.0/24`; the host owns `.1` on that network and the VM received `.134` from VMware's DHCP (evidence/00-host-vm.md, VM facts). Inside the VM, Docker Engine 29.1.3 runs the containers. For Part 2, Docker Compose 2.40.3 runs the service container directly. For Part 3, a k3d 5.9.0 cluster runs a single k3s 1.37.1 node as a Docker container on the same engine, and the Kubernetes Pods run inside that node container. At the bottom of every path is the same process: Node 22 running the course starter's `server.js`, which answers `GET /` with a JSON body (service name, message, hostname, and a request counter kept in process memory) and `GET /health` with HTTP 200.

Both environments use one container image, built once inside the VM from `app/Dockerfile`. Compose runs it from the local Docker image store; the cluster receives it through `k3d image import`, and the Deployment sets `imagePullPolicy: Never` so the cluster can only use the imported image (evidence/02-kubernetes.md, Import the same application image). No registry is involved. The image was first built as `cs554-midterm:1.0`; the CS 554 readiness experiment added a four-line toggle to `/health` and rebuilt it as `1.1`, which both environments now run. Earlier captures showing `1.0` are unchanged.

2 Docker Compose inside the VM

The Compose file declares one service, `web`, built from `../app` and configured through environment variables (`PORT`, `SERVICE_NAME`, `MESSAGE`) read from an uncommitted `.env` file whose defaults are committed as `.env.example`, with VM port 8080 published to the container's port 8080 and a health check calling `/health` every ten seconds (evidence/01-compose.md, Resolved Compose configuration). The health check deliberately targets `/health`, which the service does not count, so it never disturbs the request counter.

From inside the VM, three requests to `GET /` returned counts 1, 2, and 3 from the container hostname `864d1c1b9d87`, and `/health` returned HTTP 200 (evidence/01-compose.md, VM requests and health). From the Windows host, `curl.exe http://192.168.197.134:8080/` returned count 4 from the same container (evidence/01-compose.md, Windows host request). The host reaches the VM directly on its NAT address; VMware's NAT device routes between the host adapter and the guest, so no port

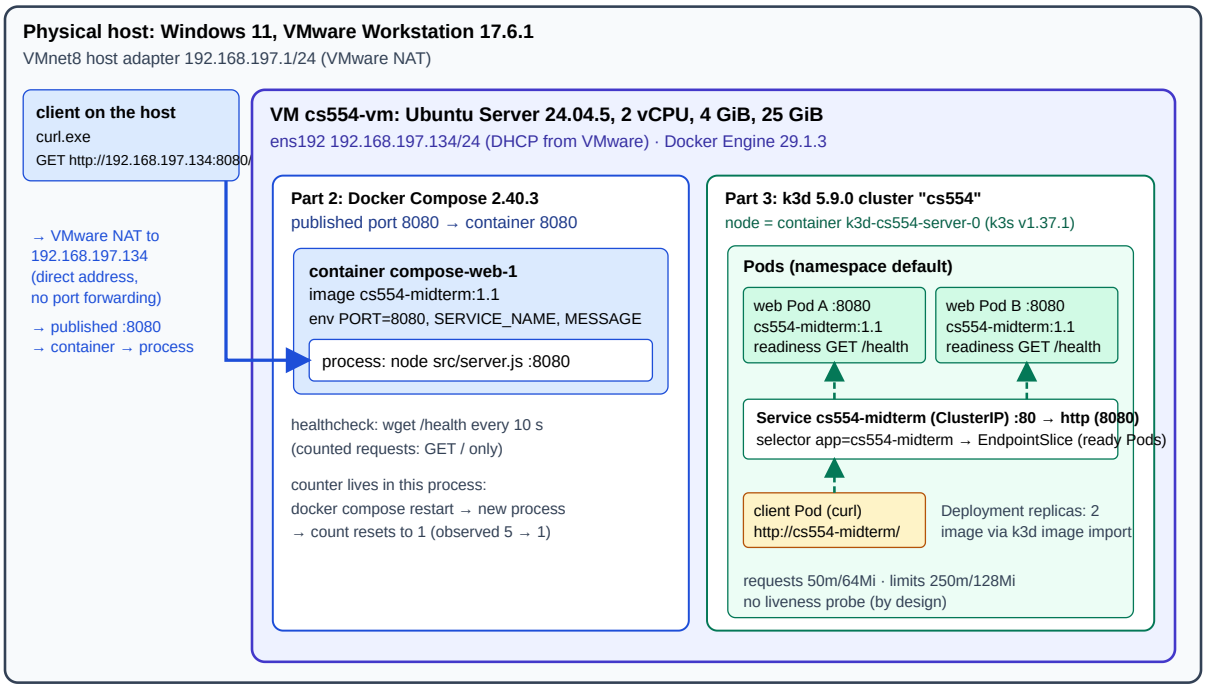


Figure 1: Host, VM, container, and application layers, with the Part 2 request path from the host (solid) and the in-cluster Kubernetes path (dashed). Ports and addresses are labeled as observed. The image tag shown is the final `1.1`; the Part 2 and Part 3 captures ran `1.0`, which differs only by the readiness toggle.

forwarding was configured. `docker compose ps` showed the container `Up (healthy)`, and the logs showed the startup line `Listening on port 8080` followed by one JSON line per request, including the periodic `/health` probes (evidence/01-compose.md, Startup and request logs).

The restart experiment is the observation the rest of the report leans on. With the count at 5, `docker compose restart` stopped and started the container: the logs recorded `Received SIGTERM; stopping.` and then a fresh `Listening on port 8080`, and the next request returned count 1 (evidence/01-compose.md, Restart and counter reset). The container kept its name and hostname; only the process inside it was new, and the counter lived in that process. Throughout, the health check's own requests appear in the logs as `/health` lines and never moved the counter. `docker compose down` removed the container and its network; the service was then brought back up for the walkthrough (evidence/01-compose.md, Cleanup demonstration).

3 Kubernetes

The cluster runs inside the same VM: k3d creates the node as a Docker container from the image `rancher/k3s:v1.37.1-k3s1`, pinned so that the server matches the installed kubectl 1.37.1 (evidence/02-kubernetes.md, Create the cluster). A Deployment named `cs554-midterm` asks for two replicas of the image, labels its Pods `app=cs554-midterm`, and a ClusterIP Service of the same name selects that label and maps port 80 to the container's named port `http` (8080). After `kubectl apply`, the Deployment reported `2/2`, both Pods were `1/1 Running`, and the Service's EndpointSlice listed two endpoints (evidence/02-kubernetes.md, Apply and two ready replicas).

Requests went through the Service in two ways, because they demonstrate different things. From a client Pod inside the cluster, six requests to `http://cs554-midterm/` by DNS name were answered by both Pods, each with its own counter; this shows the Service resolving to ready backends across replicas, though not round-robin ordering or fairness (evidence/02-kubernetes.md, In-cluster Service requests). A `kubectl port-forward svc/cs554-midterm 18080:80` session inside the VM answered three requests all from one Pod: port-forward uses the Service's selector only to choose a Pod, then tunnels to that Pod through the API server, so it proves reachability but says nothing about traffic distribution (evidence/02-kubernetes.md, VM-local Service port-forward).

Deleting one Pod showed what distinguishes Kubernetes from Compose. With Pods `mkspb` and `ndjs4` running, `kubectl delete pod` on `mkspb` was followed within three seconds by a new Pod, `2jmll`, in `ContainerCreating`, which was `Running` at three seconds and `1/1` ready at six; the ReplicaSet returned to desired/current/ready 2 and logged a `SuccessfulCreate` event (evidence/02-kubernetes.md, Automatic Pod replacement). Kubernetes replaced the Pod because the ReplicaSet controller continuously reconciles the Pods it observes against the Deployment's declared `replicas: 2`; one Pod short, it created another, without any further command. The replacement's first request counted 1 (evidence/02-kubernetes.md, Replacement process counter). The comparison with Part 2 is direct: Compose restarted the same container's process because an operator told it to, whereas Kubernetes created a different Pod, with a new name, on its own. Both lost the in-memory counter, because in both cases the process was new; what differed was who decided and what identity survived.

4 Controlled troubleshooting: the Service selector

A Service is a label query, and the EndpointSlice is the result of that query. The experiment changed one line in `service.yaml`, the selector from `app: cs554-midterm` to `app: cs554-midterm-typo`, and applied it; nothing else changed (evidence/03-selector-break.md, Exact selector change). From the client Pod, three requests to the Service by DNS name failed with `curl: (7) Failed to connect to`

`cs554-midterm port 80 ... Could not connect to server`, exit code 7. A port-forward session started fresh after the change exited with `error: timed out waiting for the condition`, and its request also failed with exit 7 (evidence/03-selector-break.md, In-cluster client symptoms; Fresh port-forward failure). The session had to be fresh: an old port-forward would have stayed pinned to the Pod it chose earlier and hidden the fault.

The observations separate the cause from everything it was not. Both Pods stayed `1/1 Running` with zero restarts and their labels unchanged; the Deployment's UID, generation, and spec were byte-identical before, during, and after; the Service's selector read `app=cs554-midterm-typo`; the EndpointSlice had no endpoints (kubectl 1.37 renders an empty endpoint set as `<unset>` and a blank `Endpoints:` line, where older versions printed `<none>`); and the Pods' logs from the moment of the change onward contained only readiness probes and not one `GET /` (evidence/03-selector-break.md, Failure diagnosis). The cause follows: no Pod carried the label the selector asked for, so the EndpointSlice controller computed an empty backend set, kube-proxy had no destination for Service traffic, and port-forward could not pick a Pod. Re-applying the correct `service.yaml` restored two endpoints, and both client paths succeeded again, with the same Pod names and zero restarts throughout (evidence/03-selector-break.md, Correct selector and recovery).

Recovery has a timing subtlety of its own. The EndpointSlice is recomputed as soon as the selector matches again, but the node's forwarding rules are rewritten by kube-proxy's own control loop, which keeps them synchronized with the API state [1], so for an instant after the object lists both backends a request can still be refused. The README's recovery step therefore retries the request and records the attempt on which it succeeded: a successful request, not a populated EndpointSlice, is the evidence that the Service is back.

Healthy Pods therefore prove nothing about reachability through a Service. Readiness is a statement about a process: it can serve. A Service's endpoints are a statement about a match: these Pods carry the labels this selector names. The two are evaluated independently, and either can be true while the other is false. A diagnosis that stops at `kubectl get pods` sees two healthy replicas and no problem; it must also compare the selector to the labels, inspect the EndpointSlice, and test a client path that depends on the match.

5 OpenStack deployment analysis

5.1 Resources and choices

Resource	Value in the class cloud	Who provides it
Image	`ubuntu-24.04`	administrator
Flavor	`m1.small`	administrator (I choose it)
Key pair	my own, created once	me
Tenant network / subnet	`private` / `private-subnet`	administrator
Router to external network	`router1` → `public`	administrator
Security group	`default` + my rules	me (rules)
Floating IP	allocated from `public`	me
User data	cloud-init: install Docker, run Compose	me

Table 1: OpenStack resources for the workload, following the class environment [2].

The server boots from the administrator's `ubuntu-24.04` image on the `m1.small` flavor with my key pair, takes a port on the administrator's `private` network and `private-subnet`, and receives a floating IP from `public`, which `router1` already connects to `private`; rules I add to the `default`

security group admit the traffic, and cloud-init user data installs Docker and starts Compose [2]. `m1.small` is enough: with the Node process running, our VM used 505 MiB of 3.8 GiB (`free -h`, evidence/00-host-vm.md). The administrator supplies the image, the flavors, `public`, `router1`, and `private`; I create the key pair, the security-group rules, the server, and the floating IP. No Cinder volume: the counter is deliberately in-process, the image is rebuilt from the repository, and nothing must outlive the instance; the ephemeral root disk suffices.

5.2 Service responsibilities

Keystone authenticates me and scopes every request to my project; the class's `devstack` cloud profile holds the credentials and endpoints, so each `openstack` command begins with a token [3, 4]. Glance serves the `ubuntu-24.04` image [5]. Nova takes the image, flavor, key pair, network, and user data, schedules and boots the instance, and installs the key pair as its SSH key [6]. Neutron creates the instance's port on `private`, routes it through `router1` to `public`, maps the floating IP onto that port, and enforces the security-group rules there [7]. Compose enters through the user data, which installs Docker and the Compose plugin, as our own VM's cloud-init did (evidence/00-host-vm.md), then clones the repository and runs `docker compose up -d`; the instance converges without a login. Cinder would matter only if data had to outlive the instance.

5.3 Request path

A client sends `GET http://<floating-ip>:8080/` to the floating IP on `public`; `router1` translates it to the instance's private address on `private-subnet` [8]; the security group on the instance's port evaluates the packet [9]; Docker's published port carries it into the container, where the Node process answers, the hop section 2 observed. The one rule the application needs is ingress TCP to port 8080 from the client's source range, distinct from SSH (TCP 22, operators only) and ICMP (diagnosis only); security groups are stateful, so the reply needs no egress rule [9]. The class exercise opened port 3000; ours listens on 8080 [4].

5.4 Failure analysis

When the server is `ACTIVE` but a client cannot reach the service, I check the layers in order. On the instance: `docker compose ps`, `curl localhost:8080/health`, `docker compose logs`; a failure here is below the network. In the VM: `ip addr` for the private address, `ss -ltnp` for 8080 bound on `0.0.0.0`, not loopback. In the cloud, via the `openstack` CLI: the floating IP's association with this instance's port, an ingress rule for TCP 8080, and `router1`'s external gateway [4, 9]. From the client: `curl -v`. The symptom names the layer: "connection refused" means nothing is listening; a timeout means a drop on the way (security group, floating IP, router); HTTP 404 means the application answered and the path is wrong. Section 4 showed the same logic one layer down, with one difference: a Service whose selector matches nothing refuses at once, so there the refusal pointed at selection, not at the application.

5.5 Operational complexity

Operating OpenStack involves more components and dependencies than deploying into a working cloud: behind one `server create` stand Keystone, Glance, Nova, and Neutron, each with its database, a shared message queue, and agents on every compute and network node that must all agree [3, 5, 6, 7]. The course's DevStack guide puts it plainly: a clean VM snapshot and a rebuild are often simpler than repairing a broken installation [10]; the restricted-network appendix

shows the tenant's side: a VM whose pulls a corporate VPN or TLS-intercepting proxy breaks, which nothing inside the VM can fix, the position of any instance behind a platform NAT or egress proxy [11]. An infrastructure problem (a Neutron agent down, a broken DevStack, an untrusted platform CA) stops every tenant and only the operator can fix it; an application problem (a missing security-group rule, a wrong published port) affects one project, and the user fixes it.

5.6 Closing comparison

A VM with Compose suffices for one instance and one process, with an operator present, manual restart acceptable, and no need to scale. In our experiment Kubernetes added a replica count enforced without intervention, a deleted Pod replaced automatically, readiness-based exclusion of an unhealthy replica from traffic, and a Service abstraction that a wrong selector can silently break, the cost of that indirection (sections 3, 4, and 6).

6 CS 554 extension: resource settings and the readiness experiment

I set requests of `cpu: 50m` and `memory: 64Mi` with limits of `cpu: 250m` and `memory: 128Mi`, producing the Burstable QoS class because the requests differ from the limits [12]. The service is a single Node process. With it running, the VM used 505 MiB of 3.8 GiB total (evidence/00-host-vm.md). Two replicas share a 2-vCPU node; the limits leave burst headroom without letting one replica starve the other.

A request is the amount the scheduler reserves on a node in order to place the Pod. A limit is the ceiling enforced at run time: CPU above the limit is throttled, and memory above the limit causes the container to be killed [13].

The readiness probe decides whether the Pod is a ready endpoint of the Service; it does not restart anything [14]. In the experiment (evidence/04-readiness-554.md), after the toggle Pod A was `0/1 Running` with `RESTARTS 0`, its EndpointSlice entry went `ready: false`, and all ten in-cluster requests went to Pod B. Pod A still answered a direct request, and its counter continued $4 \rightarrow 5 \rightarrow 6$ across failure and recovery. A liveness failure or a restart would have produced a new process and a reset counter, which did not happen.

A running Pod's readiness probe is immutable, so a per-Pod probe patch is not possible. The documented application-level toggle (`READINESS_FILE`) was the handout's permitted alternative. The Deployment template changed only in the rollout before the baseline; the experiment itself touched one file in one Pod and restored it.

References

[1] The Kubernetes Authors. Virtual IPs and service proxies. Kubernetes documentation; accessed 19 October 2026.

[2] UAH Computer Science. Openstack class environment (openstack-settings.md), 2026. Accessed 18 October 2026.

[3] OpenStack Foundation. Keystone, the openstack identity service. Accessed 18 October 2026.

[4] UAH Computer Science. Exercise 05 — deploy the same service on openstack iaas, 2026. Accessed 18 October 2026.

[5] OpenStack Foundation. Glance, the openstack image service. Accessed 18 October 2026.

[6] OpenStack Foundation. Nova, the openstack compute service. Accessed 18 October 2026.

[7] OpenStack Foundation. Neutron, the openstack networking service. Accessed 18 October 2026.

[8] OpenStack Foundation. Floating ip addresses (networking guide). Accessed 18 October 2026.

[9] OpenStack Foundation. Security groups (networking guide). Accessed 18 October 2026.

[10] UAH Computer Science. Optional: Build your own OpenStack lab with DevStack (optional-local-devstack.md), 2026. Accessed 19 October 2026.

[11] UAH Computer Science. Appendix — working behind a restricted network (restricted-networks.md), 2026. Accessed 18 October 2026.

[12] The Kubernetes Authors. Pod quality of service classes. Kubernetes documentation; accessed 18 October 2026.

[13] The Kubernetes Authors. Resource management for pods and containers. Kubernetes documentation; accessed 18 October 2026.

[14] The Kubernetes Authors. Configure liveness, readiness and startup probes. Kubernetes documentation; accessed 18 October 2026.